

Server/server_core.py

```
while self.is_running:
    try:
        self.server_socket.settimeout(1.0)
        conn, addr = self.server_socket.accept()
        client_id = f"{addr[0]}:{addr[1]}"
        self.connected_clients[client_id] = addr
        self._log(f"\n[i °£†°] í .° ...ì .ì ,í , ì °£†°: {client_id}")
        if self.log_callback:
            self.log_callback(f"[i °£†°] í .° ...ì .ì ,í ,: {client_id}")

        # £° í .° ...ì .ì ,í ,°¥... °‡ ° ì /° ° ì ì ì † °f<
        client_thread = threading.Thread(
            target=self.handle_client,
            args=(conn, addr),
            daemon=True
        )
        client_thread.start()

    except socket.timeout:
        continue
    except OSError as e:
        if self.is_running:
            print(f"ì °† ì ì... ì /°¥ °° ì : {e}")
            break

except Exception as e:
    print(f"ì °† ì /°¥ : {e}")
finally:
    if self.server_socket:
        self.server_socket.close()

def stop(self):
    """ì °† ì / ì § """
    self.is_running = False
    if self.server_socket:
        self.server_socket.close()
    print("ì °† £° ì / ì § ° ì ì µ° ° /.")

def handle_client(self, conn, addr):
    """í .° ...ì .ì ,í , ì ì † † °f<"""
    try:
        with conn:
            # STT ì † °f<° OpenAI API í ,ì ¶ °ì ì £° ì . £-,°f. ì ì ì ...°fl °ì í ì ì ì
            conn.settimeout(300.0) # 5°¶ í ì ì ì (STT ì † °f< ì £° £‡ ° /)

            # °¤...ì 4° ì .í ,°¥...ì %ì .ì °" ° „ °' ì ì § ì ,ì § í ì ,
            length_bytes = conn.recv(4, socket.MSG_PEEK)

            if len(length_bytes) == 4:
                # 4° ì .í ,£° °¤¤° 0-255 °† ì ì ì «ì ì .£‡ í '°f<ì ì , í <£,°°'. °" °
                try:
                    message_length = int.from_bytes(length_bytes, byteorder='big')
                    # °" ° „ °' ì ì § ° °‡.í µ 1KB ì .í ì .°fl °ì , 100KB ì .í °'. °" ° „ì ...°
                    if message_length > 0 and message_length < 100000:
```

```

Server/server_core.py# """
    self.handle_command(conn)
    return
except:
    pass

    # if ...
    self.receive_file(conn)

except socket.timeout:
    # if ...
    pass
except Exception as e:
    self._log(f"[ /% ] í .° ...ì .í , í , ì† °f< ì /°% ({addr}): {e}")
finally:
    client_id = f"{addr[0]}:{addr[1]}"
    if client_id in self.connected_clients:
        del self.connected_clients[client_id]
    self._log(f"[ì ¢ °£ ] í .° ...ì .í , í , ì °£†° ì ¢ °£ : {client_id}")

def handle_command(self, conn):
    """
    try:
        # STT ì† °f<°%... ì í . í ì ì ì ì f £°
        conn.settimeout(300.0) # 5°¶ í ì ì ì

        # °' ì ì§ £,,ì . ì ì (4°° ì .í ,)
        length_bytes = conn.recv(4)
        if len(length_bytes) != 4:
            return

        message_length = int.from_bytes(length_bytes, byteorder='big')

        # °' ì ì§ °‡,°<, ì ì
        message_data = b""
        while len(message_data) < message_length:
            chunk = conn.recv(min(BUFSIZE, message_length - len(message_data)))
            if not chunk:
                return
            message_data += chunk

        if len(message_data) < message_length:
            return

        # JSON í ì -
        try:
            message = json.loads(message_data.decode('utf-8'))
            command = message.get("command")
            data = message.get("data", {})

            # °" ° „ ì† °f<
            response = self.process_command(command, data, conn)

            # ì ° μ ì ì ì
            response_json = json.dumps(response, ensure_ascii=False)

```

Server/server_response.py

```

response_bytes = response_json.encode('utf-8')
response_length = len(response_bytes)

conn.sendall(response_length.to_bytes(4, byteorder='big'))
conn.sendall(response_bytes)

except json.JSONDecodeError as e:
    response = {"status": "error", "message": f"JSON í ì - ì /°¥ : {e}"}
    response_json = json.dumps(response, ensure_ascii=False)
    response_bytes = response_json.encode('utf-8')
    conn.sendall(len(response_bytes).to_bytes(4, byteorder='big'))
    conn.sendall(response_bytes)
except Exception as e:
    print(f"° ° „ ì† °f< ì /°¥ : {e}")

def process_command(self, command, data, conn):
    """° ° „ ì† °f< ° ... ì °í """
    try:
        if command == "login":
            return self.handlers.handle_login(data)
        elif command == "convert_audio":
            return self.handlers.handle_convert_audio(data)
        elif command == "save_text":
            return self.handlers.handle_save_text(data)
        elif command == "manage_folder":
            return self.handlers.handle_manage_folder(data)
        elif command == "load_file":
            return self.handlers.handle_load_file(data)
        elif command == "save_score":
            return self.handlers.handle_save_score(data)
        elif command == "generate_quiz":
            return self.handlers.handle_generate_quiz(data)
        elif command == "generate_single_quiz":
            return self.handlers.handle_generate_single_quiz(data)
        elif command == "load_quizzes":
            return self.handlers.handle_load_quizzes(data)
        elif command == "list_files":
            return self.handlers.handle_list_files(data)
        elif command == "analyze_text":
            return self.handlers.handle_analyze_text(data)
        else:
            return {"status": "error", "message": f"ì ì ì ° ° ° „ : {command}"}
    except Exception as e:
        return {"status": "error", "message": str(e)}

def receive_file(self, conn):
    """í ì ... ì ì (€,°ì ì· í °ì í ì% )"""
    try:
        # í ì ...° ì ì
        filename_data = b""
        while True:
            chunk = conn.recv(1)
            if not chunk:
                return False
            if chunk == b'\0':

```

Server/server_core.py

```
filename_data += chunk

try:
    filename = filename_data.decode('utf-8')
except UnicodeDecodeError:
    filename = filename_data.decode('latin-1', errors='ignore')

if not filename:
    return False

print(f"í ì ... ì ì ì ì : {filename}")

# ì / ° „ ì í , ì ì ì
conn.sendall(b"READY")

# í ì ... í <£, ° ì ì
file_size_data = b""
while True:
    chunk = conn.recv(1)
    if not chunk:
        return False
    if chunk == b'\0':
        break
    file_size_data += chunk

try:
    file_size_str = file_size_data.decode('utf-8')
    file_size = int(file_size_str)
except ValueError:
    return False

# í ì ... ° ° ì . í ° ì ì
file_path = os.path.join(RECEIVE_DIR, os.path.basename(filename))
received_size = 0

with open(file_path, 'wb') as f:
    while received_size < file_size:
        remaining = file_size - received_size
        chunk_size = min(BUFSIZE, remaining)
        data = conn.recv(chunk_size)
        if not data:
            return False
        f.write(data)
        received_size += len(data)

print(f"í ì ... ì ì ì °£ : {file_path}")

# ì -£‡µ ì í , ì ì ì
conn.sendall(b"SUCCESS")
return True

except Exception as e:
    print(f"í ì ... ì ì ì /°¥ : {e}")
    try:
```

```
Server/server.py:105: OSError: [Errno 100] Connection refused
    except:
        pass
    return False
```